
SEC Form 4 Data Pipeline

Technical Documentation

Module IT for Finance I

University TH Köln - Campus Südstadt

Group Group 01

Members Umeyr Özel, Emil Kloz, Kenan Mert Biberici, Kevin Kader

Supervisors Prof. Dr. Felix Miebs, Prof. Dr. Tobias Schlüter

Date February 2026

1. Introduction

Whenever corporate insiders - officers, directors, anyone with more than 10% ownership - buy or sell shares in their own company, they have to report it to the SEC through a Form 4 filing. All of that is public record. The idea is simple: if the CFO is quietly selling off stock, outside investors probably want to know. Lakonishok and Lee (2001) looked at this kind of data in aggregate and found that it does reveal patterns regular investors would miss.

What we built is a Python pipeline that grabs all the Form 4 bulk data the SEC has published for 2025, cleans it up, loads it into MySQL, checks it for obvious errors, and then generates monthly rankings of insider buying and selling per company. Charts, CSV tables, a PDF report - all automated, no manual steps. You can run it twice and nothing breaks because the import is idempotent: old data gets deleted before new data goes in. What follows covers the data itself, how we built the pipeline, what our validation catches, and what gave us trouble along the way.

2. Data Understanding

The SEC puts out quarterly data dumps as ZIP archives. Inside each one there are several TSV files - tab-separated, not comma-separated - covering every Form 4 filing from that quarter. The files follow a relational structure, so they can be joined together for analysis.

2.1 File Structure

Six TSV files per quarter. Table 1 lists them along with the columns we actually use in our pipeline.

Table 1: Overview of SEC Form 4 data files

File	Description	Key Columns
SUBMISSION.tsv	Filing metadata and issuer info	ACCESSION_NUMBER, FILING_DATE, ISSUER_CIK, ISSUER_NAME
REPORTINGOWNER.tsv	Reporting insider info	ACCESSION_NUMBER, RPTOWNERCIK, RPTOWNERNAME
NONDERIV_TRANS.tsv	Non-derivative transactions	ACCESSION_NUMBER, TRANS_DATE, TRANS_CODE, TRANS_SHARES, TRANS_PRICEPERSHARE
NONDERIV_HOLDING.tsv	Non-derivative holdings	ACCESSION_NUMBER, SHRS_OWND_FOLWNG_TRANS
DERIV_TRANS.tsv	Derivative transactions	ACCESSION_NUMBER, TRANS_DATE, TRANS_CODE, SHARES
DERIV_HOLDING.tsv	Derivative holdings	ACCESSION_NUMBER

Almost everything the SEC gives you arrives as raw strings. `ACCESSION_NUMBER` looks like it could be a number, but it has leading zeros and dashes in it, so you can't treat it as an integer - it's a `VARCHAR`. Dates show up as `DD-MON-YYYY` text (like "15-JAN-2025"), which pandas won't parse on its own. `TRANS_SHARES` and `TRANS_PRICEPERSHARE` are strings too and need to become `DECIMAL`. The reporting-owner relationship field is one long combined string that we had to split apart into separate boolean columns.

2.2 Relationships and Join Keys

Everything connects through `ACCESSION_NUMBER`. One submission can have multiple transactions underneath it - say an insider reports five different trades in a single filing. Classic one-to-many between `SUBMISSION` and the transaction tables. When we group by company in the evaluation, we use `ISSUER_CIK`, not the company name. The same company sometimes shows up with slightly different spelling across filings, but the `CIK` stays the same.

2.3 Data Quality Observations

The data is not exactly clean. Plenty of transactions come in with NULL prices or share counts - turns out that is normal when the trade is something like a gift where no money changes hands. Derivatives are messier - a lot of them don't have a price at all because the instrument is an option. The reporting owner data was another time sink. It sits in its own file (REPORTINGOWNER.tsv) and needs to be merged into the submission table, which gets tricky when one filing lists two or three owners and you need aggregation logic to collapse them into a single row.

3. Pipeline Architecture

We split the work into six phases that run one after another: download, parse, prepare, import, validate, evaluate. config.py holds all the shared settings in one place, and main.py orchestrates the phases in sequence. Figure 1 gives an overview.

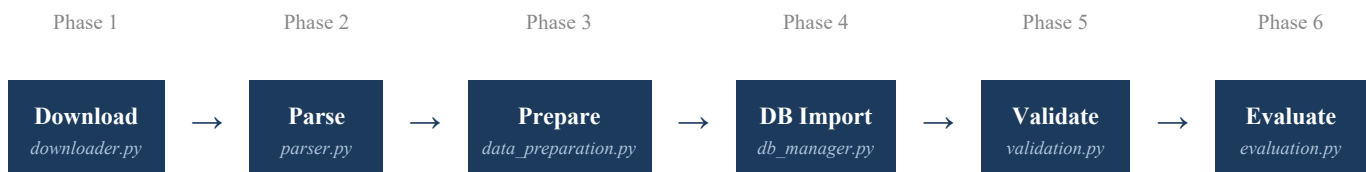


Figure 1: Pipeline data flow

3.1 Module Overview

Table 2: Module responsibilities

Module	Responsibility
config.py	Loads .env credentials, defines paths, constants, and DB settings
downloader.py	Scrapes the SEC website for available quarters, downloads ZIP files with retry logic and rate limiting
parser.py	Extracts TSV files from the ZIPs, reads them into pandas DataFrames with encoding fallback (UTF-8, then Latin-1)
data_preparation.py	Cleans column names, converts dates and numbers, merges reporting owner data into submissions, adds metadata columns
db_manager.py	Creates the MySQL schema and all seven tables, handles the idempotent delete-then-insert import with per-table transactions
validation.py	Runs eight plausibility checks on transaction data and stores results both inline and in a dedicated log table
evaluation.py	Queries validated data from the database, generates monthly ranking charts, CSV exports, overview charts, and a PDF report

3.2 Database Schema

Seven tables, all under the schema 'group01'. submissions is the parent table, and nonderiv_trans, nonderiv_holdings, deriv_trans, and deriv_holdings reference it. At startup the pipeline attempts to add foreign key constraints with ON DELETE CASCADE via ALTER TABLE, but the university server does not grant the REFERENCES privilege, so the statements fail gracefully and we fall back to enforcing insertion and deletion order in code instead. Then there's validation_log for check results and pipeline_log as an audit trail. pipeline_log never gets wiped during re-imports - that is the whole point of having it.

3.3 Configuration and Credentials

Passwords go into a .env file that is excluded from version control. config.py picks them up through python-dotenv, so nothing sensitive ends up in the source code. We put an .env.example in the repository so you know which variables need to be filled in.

4. Validation Logic

After the import we run validation on every transaction row. The assignment required four checks, and we wrote four more on top of that because the mandatory ones alone don't catch everything.

4.1 Implemented Checks

Table 3: Validation checks and their rationale

#	Check Name	Description	Type
1	Mandatory Fields	ACCESSION_NUMBER and TRANS_DATE must not be NULL	Mandatory
2	Negative Prices	PRICE_PER_SHARE cannot be negative (NaN is acceptable)	Mandatory
3	Negative Shares	SHARES cannot be negative	Mandatory
4	Date Logic	TRANS_DATE must not be later than FILING_DATE (requires JOIN)	Mandatory
5	Valid Trans. Code	TRANS_CODE must be one of the known SEC Form 4 codes	Bonus
6	Future Date	TRANS_DATE must not be in the future	Bonus
7	Reasonable Price	PRICE_PER_SHARE at or above \$1,000,000 is flagged	Bonus
8	Orphan Records	Transaction must have a matching submissions entry (LEFT JOIN)	Bonus

4.2 Result Storage

We store results in two places. On the transaction row itself there's an `is_valid` flag (0 or 1) plus a `validation_flags` text column - failed check names get concatenated with semicolons in there. Querying clean data in SQL is just `WHERE is_valid = 1`. On top of that we write to a separate `validation_log` table, one row per failed check, with a `details` column in plain text. That second table is mostly for debugging - if a row gets flagged and you want to know why, you look there.

The evaluation phase only uses rows where `is_valid = 1`. Anything flagged gets filtered out before the aggregation runs.

5. Automation and Reproducibility

5.1 Dynamic Quarter Discovery

Nowhere in the code do we hardcode Q1 through Q4. The downloader scrapes the SEC data page, finds ZIP links that match the target year, and pulls down whatever's there. If Q3 shows up on the website tomorrow, the next run just picks it up - no code changes needed.

5.2 Idempotent Re-execution

You can re-run the whole pipeline without getting duplicates. Before importing, all rows for that quarter get wiped - child tables first (transactions, holdings), then submissions. `pipeline_log` is the only table we don't touch, because it's supposed to keep a record of every run.

5.3 Error Handling and Retry Logic

Downloads and database writes both have retries baked in. A 429, 500, or 503 from the SEC triggers exponential backoff. We also pause 200ms between requests - the SEC asks for at least 100ms. On the database side, lock wait timeouts come up more than you'd think, mostly when other groups on the same server are importing at the same time. Same solution: retry with increasing wait. Each table also gets its own transaction, so locks don't pile up across the whole database.

6. Reflection

Most of the problems we ran into had nothing to do with the lectures - they came from the infrastructure and the data itself.

6.1 Shared Database Server

The shared MySQL server was by far the worst. All course groups run on one instance, so when two or three of us import at the same time, you get lock wait timeouts constantly. We originally had the entire import in one big transaction. Bad idea - it locked all tables at once and the server just killed it. Now each table gets its own transaction, lock timeout is set to 120 seconds, and on failure the code backs off and retries. Not perfect, but good enough.

6.2 Missing REFERENCES Privilege

The university server does not grant the REFERENCES privilege. Our code attempts to add foreign key constraints at startup via ALTER TABLE, but when the server denies it, the exception is caught and the pipeline continues without them. So we enforce the order in code instead: submissions go in first, come out last. A real FK with CASCADE would simplify deletion to a single statement, but the manual ordering works just as well since we are the only ones writing to this schema.

6.3 Connection Stability

Halfway through chart generation, SQLAlchemy would sometimes crash with 'MySQL server has gone away'. Idle connections get dropped by the server after a while. pool_pre_ping fixed most of it - tests the connection before using it. pool_recycle at 280 seconds forces a refresh before the timeout hits. For the evaluation queries specifically we added a retry that throws away the engine and builds a new one. Not pretty, but it works.

6.4 Reporting Owner Merge

Reporting owner data comes in a separate file (REPORTINGOWNER.tsv) but we need it in the submissions table. The catch: one filing sometimes lists two or three owners, like someone who's both a director and a ten-percent holder. We group by ACCESSION_NUMBER, grab the first owner's CIK and name, and OR the boolean flags together. So is_director ends up as 1 if any of the listed owners has that role. Keeps the schema flat without losing any of the ownership information.

6.5 Possible Improvements

With more time we'd speed up validation - it updates rows one by one right now, and a bulk UPDATE via temporary table would help a lot once you're past a hundred thousand rows. We thought about only downloading quarters that actually changed since the last run, but didn't get to it. Cross-checking CIKs against the SEC's EDGAR company index would've been another nice check, but at some point you have to stop adding things.

References

- Lakonishok, J., & Lee, I. (2001). Are insider trades informative? *The Review of Financial Studies*, 14(1), 79–111. <https://doi.org/10.1093/rfs/14.1.79>
- U.S. Securities and Exchange Commission. (2025). *Insider transactions data sets*. Retrieved February 2026, from <https://www.sec.gov/data-research/sec-markets-data/insider-transactions-data-sets>
- U.S. Securities and Exchange Commission. (n.d.). *Insider transactions and forms 3, 4, and 5* [PDF]. <https://www.sec.gov/files/forms-3-4-5.pdf>
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>
- Bayer, M. (2012). SQLAlchemy. *The architecture of open source applications, Volume II: Structure, scale, and a few more fearless hacks*. <https://aosabook.org/en/v2/sqlalchemy.html>